End-to-End MLOps for Customer Churn Prediction: From Training to Drift-Aware Monitoring with a Reproducible Container Stack

Dan Waititu

Lucerne University of Applied Sciences and Arts (HSLU)
MSc Module: Artificial Intelligence
GitHub link: https://github.com/DanYT2/MLOPS_HSLU_project
danwwaititu@gmail.com

***Abstract*—This report describes the design, implementation and operation of an end-to-end machine-learning operations (MLOps) pipeline for binary customer-churn prediction on the well-known IBM Telco dataset. The system pairs a dual-model gradient-boosting ensemble (LightGBM [1] and XGBoost [2]) with Bayesian hyperparameter optimisation via Optuna [3], five-fold stratified cross-validation, and an MLflow [4] registry-backed promotion workflow. Predictions are served through a FastAPI [5] web service inside a Docker Compose [6] topology that includes Postgres, Grafana and a replay-based monitoring worker which writes Evidently [7] drift and quality metrics to a relational store visualised by an eight-panel Grafana dashboard. A multi-workflow GitHub Actions [8] pipeline enforces linting, testing, coverage, container builds with SLSA provenance [9] and weekly secret scanning. The artefact closes the MLOps loop end to end on a single developer workstation and is reproducible by a single docker compose up invocation. The contribution of this work is not a new modelling result but a documented, integrated reference implementation of the operational practices distilled by Sculley *et al.* [10] and Paleyes *et al.* [11].**

***Index Terms*—MLOps, customer churn, gradient boosting, model registry, drift detection, continuous integration, reproducibility.**

I. Introduction

Customer churn—the rate at which existing customers terminate their relationship with a service provider—is a recurring concern in subscription industries because acquiring a new customer is materially more expensive than retaining an existing one [12]. Predictive churn modelling allows targeted retention interventions, but the value of such a model is realised only once it is deployed, monitored and re-trained inside an organisational workflow.

A long line of empirical work has documented that the modelling code is the smallest part of a production ML system; the much larger surface area consists of data plumbing, configuration management, monitoring infrastructure, serving infrastructure and governance controls [10], [11]. The discipline of MLOps emerged to address this imbalance by transferring established software-engineering practices—continuous integration, infrastructure as code, observability, version control of artefacts—into the ML lifecycle.

This report documents an end-to-end MLOps reference implementation built for the HSLU Artificial Intelligence module. The system targets the IBM Telco customer-churn dataset and integrates ten open-source tools into a single DOCKER COMPOSE stack. The deliverables are:

1) a training pipeline that performs Bayesian hyperparameter optimisation followed by stratified cross-validation of a dual LightGBM/XGBoost ensemble, logging every artefact to MLflow;
2) a FastAPI serving layer that loads the currently aliased CHAMPION model from the MLflow registry at start-up and applies the same preprocessing as training;
3) a Postgres-backed monitoring loop that replays held-out and unseen data through the serving API, computes Evidently drift and classification metrics in batches, and visualises the results in an eight-panel Grafana dashboard with optional synthetic drift injection;
4) a five-workflow GitHub Actions CI/CD pipeline that runs Ruff, Pytest with a coverage gate, container builds with SBOM and SLSA v1 provenance attestations, weekly Gitleaks secret scans, and (optionally) an end-to-end training run.

The remainder of this report is organised as follows. Section II reviews the relevant background. Section III presents the system architecture and tool inventory. Sections IV–VII describe the data, training, serving and monitoring components. Section VIII discusses the testing, CI/CD and supply-chain controls. Sections IX–X reflect on lessons learnt and propose future work; Section XI concludes.

II. Background and Related Work

A. Gradient-boosted decision trees

Gradient-boosted decision tree (GBDT) ensembles remain the state-of-the-art family of algorithms for tabular classification problems. XGBoost [2] introduced a scalable implementation with weighted-quantile sketching and

sparsity-aware split finding, while LightGBM [1] subsequently improved training-time efficiency through gradient-based one-side sampling and exclusive feature bundling. Because the two implementations make different inductive choices (level-wise vs. leaf-wise growth, histogram-binning details, regularisation form), averaging their predictions is a common variance-reduction strategy and is the approach adopted in this work.

B. Bayesian hyper-parameter optimisation

Random and grid search waste budget on uninformative regions of the search space. Bergstra *et al.* [13] introduced the tree-structured Parzen estimator (TPE) sampler, which fits two density estimators over the history of trials and proposes new configurations that maximise the expected improvement ratio. Optuna [3] provides a production-ready TPE implementation with pruning, persistent SQLite-backed studies and a define-by-run search-space API; the pipeline in this report uses Optuna for both LightGBM and XGBoost search spaces.

C. MLOps maturity

Sculley *et al.* [10] characterised the "hidden technical debt" of production ML systems—glue code, configuration sprawl, undeclared consumers, feedback loops—and motivated the operational disciplines that have since been collected under the MLOps banner. The ML Test Score [14] proposed a rubric covering data, model, infrastructure and monitoring tests. Polyzotis *et al.* [15] surveyed the data-lifecycle challenges of production ML, and Paleyes *et al.* [11] catalogued recurring deployment failure modes from industry case studies. The architecture described in Section III explicitly maps to a subset of these recommendations.

D. Drift detection

Concept drift—a change in the joint distribution $P(X, Y)$—degrades the calibration and discrimination of deployed classifiers and is the dominant operational risk for tabular models [16]. Practical detectors operate at the *feature* level (e.g., Population Stability Index, Kolmogorov–Smirnov, Wasserstein distance) or the *prediction* level (drift in the score distribution). The Evidently [7] library, used in this work, packages a battery of such tests behind a single report API and is the source of the drift metrics emitted by the monitor service described in Section VII.

III. System Architecture and Tool Stack

Figure 1 shows the six Docker Compose services that make up the runtime: the FastAPI prediction `api`, the `mlflow` tracking server, a `postgres` metrics database, the `grafana` visualisation server, an `adminer` database UI for ad-hoc inspection, and the `monitor` replay worker. Training is performed in a separate container (or directly on the host during development) that reads the dataset and writes runs into the MLflow tracking store. The serving and training stages share a single artefact contract—a registered model named `CustomerChurnEnsemble` carrying an alias `@champion`—which decouples the model's identity from its underlying version number and is the single point of integration between training and serving.

a) Network boundary: An onboarding pitfall worth emphasising early is that two URL spaces coexist: inside the Compose network services address each other by service name (e.g., the API uses `MLFLOW_TRACKING_URI=http://mlflow:5001`), while a developer on the host machine reaches the same services via published `localhost` ports (5001, 8000, 3000, 8080). Misconfiguring this boundary is the most common cause of first-run errors.

b) Tool inventory: Table I lists every third-party tool in the stack together with the role it plays in this project. Versions are pinned in `pyproject.toml` and the `docker-compose.yml` file.

IV. Data and Feature Engineering

The pipeline consumes the IBM Telco customer-churn dataset, expanded by augmentation to roughly 600 k rows for training and a separately provided unlabelled test set. The schema contains a mixture of numerical attributes (`tenure`, `MonthlyCharges`, `TotalCharges`, `SeniorCitizen`) and categorical attributes describing services subscribed (phone, internet, streaming, add-on protection) plus contractual and payment information. The target is the binary indicator `Churn`.

a) Derived features: Two derived features are added to the raw schema. `AvgMonthlyCharge` is computed as `TotalCharges/tenure` (with zero-tenure guard); it disentangles total revenue from tenure length and is therefore a less collinear signal than its components (Figure 3). `TotalServices` is the count of subscribed add-on services per customer; it summarises the breadth of the customer's relationship with the operator in a single integer.

b) Encoding: Categorical attributes with more than two levels are one-hot encoded; binary categorical attributes (e.g., `Partner`, `PaperlessBilling`) are mapped to {0,1}; the `gender` column is dropped as it carries no discriminative signal on this dataset and is excluded for fairness considerations. The final feature list is persisted alongside the model in the MLflow registry to guarantee column-order parity at serving time.

c) Split: A stratified split preserves the empirical churn rate in every cross-validation fold; the monitor service later re-uses the same stratified split, holding out 20 % of the training set as a "reference" distribution against which incoming batches are compared.

V. Training Pipeline

The training stage is implemented in `train.py` and is visualised in Figure 5. It proceeds in five logical phases.

a) Phase 1 – Hyper-parameter optimisation: Two independent Optuna studies (one per base learner) are run for a configurable number of trials, persisting to `optuna_studies.db` so that interrupted runs can resume

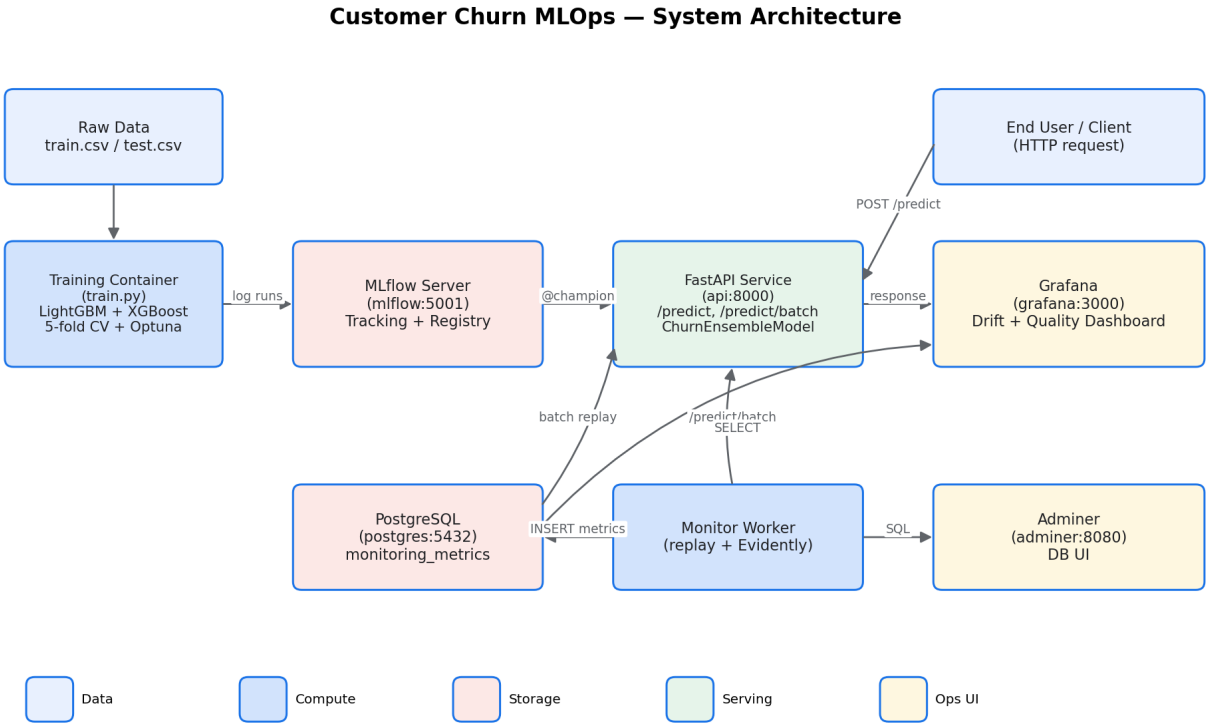


Fig. 1: System architecture. Boxes are Docker Compose services; arrows show data and control flow. Service-name DNS resolves inside the Compose network (e.g., `mlflow:5001`); host ports are published for the UIs (e.g., `localhost:5001` for MLflow, `localhost:3000` for Grafana).

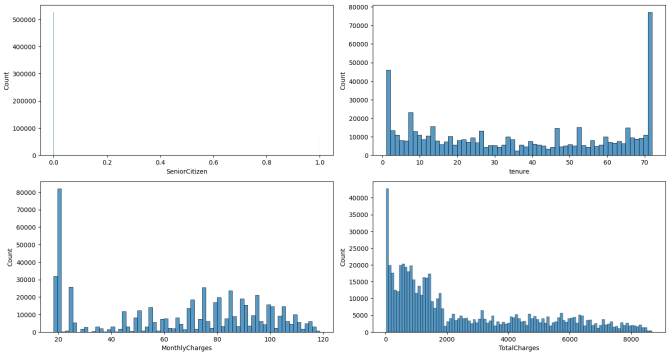


Fig. 2: Marginal distributions of the four numeric attributes. `tenure` is bimodal (recently-joined customers and long-tenured stayers), `MonthlyCharges` clusters around the price points of the bundled tariffs, and `TotalCharges` is heavily right-skewed.

and so that the search history can be inspected post-hoc. For each trial Optuna samples a candidate configuration from the search space (e.g., `learning_rate`, `num_leaves`, `max_depth`, regularisation strengths) and scores it with a single hold-out evaluation of the validation AUC.

b) Phase 2 – Stratified cross-validation: The best hyper-parameter set per base learner is then refit five times under stratified five-fold cross-validation. Each fold trains both a LightGBM and an XGBoost model on the same training partition, yielding ten fold-level models in total. Validation AUC, F1, accuracy, precision, recall and log-loss are recorded per fold.

c) Phase 3 – Ensemble assembly: The ten fold models are wrapped inside a custom `pyfunc` class (`ChurnEnsembleModel`) whose `predict()` method (i) reindexes the incoming feature frame to the persisted column order, (ii) averages the five LightGBM probabilities, (iii) averages the five XGBoost probabilities, and (iv) returns the mean of the two means. Wrapping all fold models inside a single registry entity keeps the registry surface area small and guarantees atomic promotion.

d) Phase 4 – MLflow logging: Each training run produces a parent MLflow run with ten nested child runs (one per fold-learner combination). The parent run records aggregate metrics and the serialised ensemble; child runs carry per-fold metrics, the per-fold confusion matrix as a PNG artefact (Figures 9a and 9b), and the per-fold feature-importance plot.

e) Phase 5 – Registry promotion: After successful completion the parent run is registered under the name `CustomerChurnEnsemble`; the operator (or, in future work, an automated gate) attaches the `@champion` alias to the new version, which is the version subsequently loaded by the serving container at start-up.

TABLE I: Tool inventory and per-tool role in the pipeline.

Tool	Version	Role in this project
LightGBM	4.6.0	First gradient-boosting learner; 5 fold models per pipeline run.
XGBoost	3.2.0	Second gradient-boosting learner averaged with LightGBM at inference.
scikit-learn	1.x	Stratified K-fold splitter, classification metrics, baseline utilities.
Optuna	4.7.0	Bayesian (TPE) hyper-parameter optimisation; studies persisted in SQLite.
MLflow	3.10.0	Experiment tracking, artefact store, model registry and alias-based promotion.
FastAPI	0.135.1	ASGI web framework hosting the prediction endpoints.
Pydantic	2.x	Request/response schema validation at the API boundary.
Uvicorn	–	ASGI server in the API container.
joblib	–	Serialisation of the per-fold sklearn-compatible boosters inside the pyfunc wrapper.
Evidently	0.7.21	Drift and classification quality reports computed batch-by-batch by the monitor.
PostgreSQL	16	Storage backend for the `monitoring_metrics` table read by Grafana.
psycopg / psycopg2	3.3.3 / 2.9.12	Python clients used by the monitor and Grafana datasource respectively.
Grafana	11.3.0	Eight-panel monitoring dashboard, provisioned from JSON.
Adminer	5	Web UI for ad-hoc inspection of the metrics database.
pandas / NumPy	–	Data loading and tensor manipulation throughout.
matplotlib / seaborn	–	Feature importance plots, confusion matrices and EDA figures.
Docker, Docker Compose	–	Container build and multi-service orchestration.
uv	–	Fast dependency resolver used by all CI workflows.
Ruff	0.15.5	Linting (gating) and formatting (advisory) in CI.
Pytest, pytest-cov	8.3.0 / 5.0.0	Test runner and coverage measurement with a 50% floor.
httpx	0.28.0	In-process HTTP client used by the FastAPI test client.
pip-audit	2.7.3	Supply-chain vulnerability scanning of the resolved environment.
Gitleaks	–	Pre-merge and weekly secret scanning, reporting to GitHub's code-scanning UI.
GitHub Actions	–	Five workflows: `ci`, `docker`, `release`, `secret-scan`, `train`.
CodeQL	–	Static-analysis security scanning via GitHub's default setup.

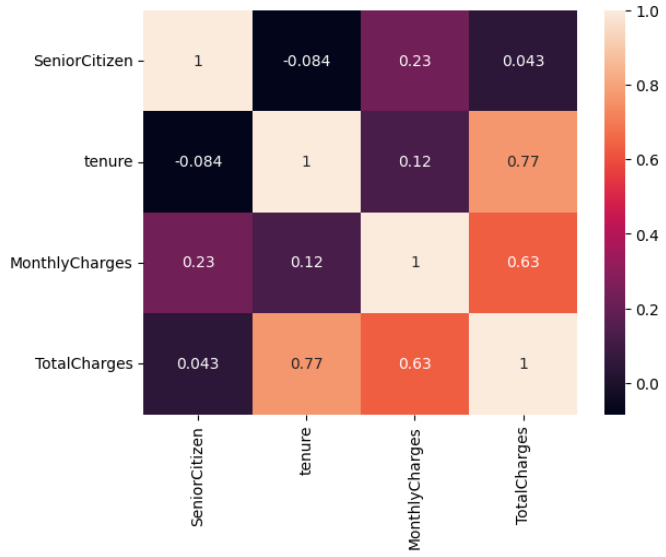


Fig. 3: Pearson correlation between numeric features. The strong correlation between `tenure` and `TotalCharges` (0.77) motivates the derived feature `AvgMonthlyCharge` introduced in Section IV.

f) Score distribution: Figure 12 shows the distribution of ensemble churn probabilities on the held-out test set. The mass is concentrated near zero (the majority class) with a long tail towards one, justifying the use of AUC and log-loss as primary metrics rather than raw accuracy.

VI. Serving Layer (FastAPI)

The serving layer is a FastAPI application (`web_service.py`) that loads the model at start-up and exposes four endpoints: `GET /` (health), `POST /predict` for a single customer, `POST /predict/batch` for an array of customers, and `GET /model/info` which returns the loaded MLflow version metadata. Requests and responses are typed by Pydantic models defined in `schemas/schemas.py`; six enumerations enforce the legal values of categorical fields (e.g., `Contract`, `PaymentMethod`, `InternetService`) and bounded numeric constraints catch out-of-range `tenure` or charges before they reach the model.

a) Train/serve parity: A frequent failure mode of deployed classifiers is feature-engineering drift between training and serving. This system addresses the risk by re-implementing the training-time transformations inside the API in a function whose only inputs are the validated Pydantic payload and the persisted feature list. The function performs (i) enum-to-binary mapping for `YesNo` fields, (ii) one-hot encoding of multi-level categoricals, (iii) re-derivation of `AvgMonthlyCharge` and `TotalServices`, and (iv) reindexing onto the canonical column order persisted with the model. Defects introduced by hand-maintaining this parity are discussed in Section IX.

b) Container path resolution: A small helper translates the host-relative MLflow artefact paths recorded inside `mlruns/` into the in-container paths used at run time. Without this translation the API would attempt to read

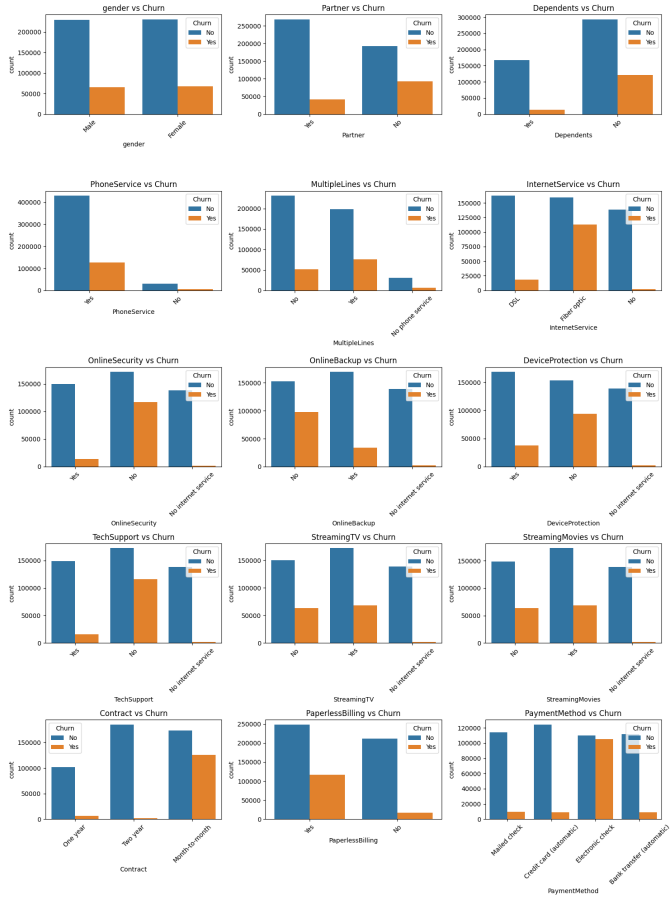


Fig. 4: Churn rate by categorical attribute. Month-to-month contracts, fibre-optic internet and electronic-check payments show markedly elevated churn relative to other levels.

artefacts at the filesystem location they happened to occupy on the developer's laptop.

VII. Monitoring and Drift Detection

The monitoring component is a long-running container (`monitor.py`) that replays both the labelled hold-out ("reference") and the unlabelled test stream through the serving API in fixed-size batches at a fixed cadence (defaults: batch size 1 000, interval 10 s, configurable through environment variables). For each batch the worker (i) collects the churn probabilities returned by the API, (ii) constructs an Evidently `Report` that includes a dataset-level drift metric, a per-column drift metric on the prediction distribution, and (when labels are available) classification quality metrics, and (iii) inserts a single row into the `monitoring_metrics` table in Postgres. Eight Grafana [17] panels read from that table and refresh on a five-second interval.

a) Drift definition: The dataset-level metric reports the number of input columns whose distribution has drifted relative to the reference fold, using Evidently's default per-type detectors (Jensen– Shannon distance for numeric features, Wasserstein for ordinal, chi-squared for categorical). The prediction-drift metric uses Population Stability Index on the predicted-probability histogram. A column is considered drifted when its detector's p-value falls below the standard 0.05 threshold; the share of drifted columns is the primary drift health indicator displayed in red on the dashboard.

b) Synthetic drift injection: For demonstration and stress testing, the worker can inject synthetic drift into the test stream through three modes (`gradual`, `sustained`, `cycle`) controlled by environment variables. The injector amplifies a configurable subset of numerical features by an intensity that varies over batches; the resulting drift signal then propagates through the monitor, Postgres and Grafana so that the visualisations become observably reactive.

VIII. Testing, CI/CD and Supply-Chain Security

A. Test suite

The pytest suite in `project/tests/` is organised in three layers. `test_schemas.py` exercises the Pydantic validation rules to catch schema regressions (eight tests over enum membership, field constraints and response serialisation). `test_web_service.py` drives the FastAPI app through `TestClient`, substituting the registry-backed model with a `_StubChurnModel` fixture that always returns probability 0.42; this isolates the endpoint, preprocessing and response-shaping logic from MLflow availability. `test_monitor.py` verifies the synthetic drift injector's intensity scheduling and the CSV-to-API feature transformation. A 50 % coverage floor is enforced in CI; `train.py` and the Grafana provisioning are intentionally omitted from the measured surface because they are exercised by the `train.yml` workflow rather than by pytest.

B. CI/CD pipeline

Figure 18 summarises the five GitHub Actions workflows.

- **ci.yml**—runs on every push and PR to `main`; executes Ruff (lint as hard gate, format check advisory), Pytest with a coverage gate and uploads a coverage XML artefact.
- **docker.yml**—runs on push to `main`, semver tags and PRs; builds the three production images (`api`, `mlflow`, `monitor`) and pushes them to GHCR with a CycloneDX SBOM and SLSA v1 provenance attestation [9].
- **release.yml**—re-tags GHCR images and publishes a GitHub release on each semver tag.
- **secret-scan.yml**—runs Gitleaks on every push, every PR to `main` and on a weekly cron (Monday 05:00 UTC); findings appear as SARIF reports in GitHub's code-scanning UI and block PRs.
- **train.yml**—manual dispatch; performs an end-to-end training run with a temporary MLflow server, uploads `submission.csv` and the entire `mlruns` directory as workflow artefacts so that registered models can be inspected without rebuilding the environment locally.

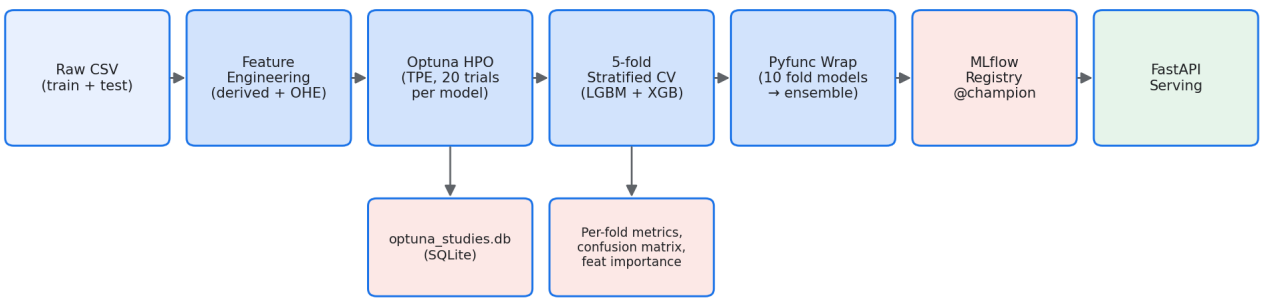


Fig. 5: Training pipeline. Raw CSV is feature-engineered, the hyper-parameters of each base learner are tuned by Optuna, five-fold stratified cross-validation produces ten fold models (five LightGBM + five XGBoost) that are packaged into a single `pyfunc` ensemble and registered in MLflow under the alias `@champion`.

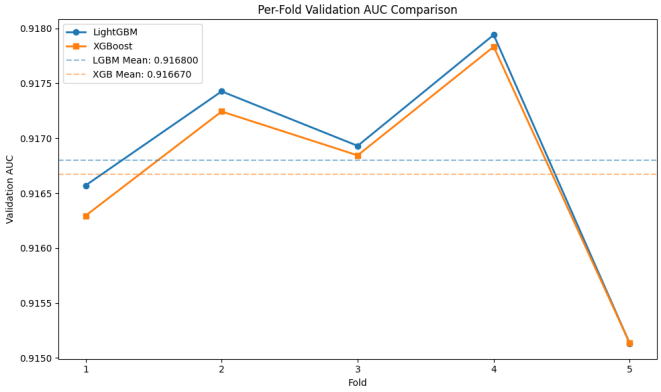


Fig. 6: Per-fold validation AUC of LightGBM and XGBoost. The two learners track each other closely (mean AUC differs in the fourth decimal), supporting the use of a simple equal-weight average rather than a learned stacker.

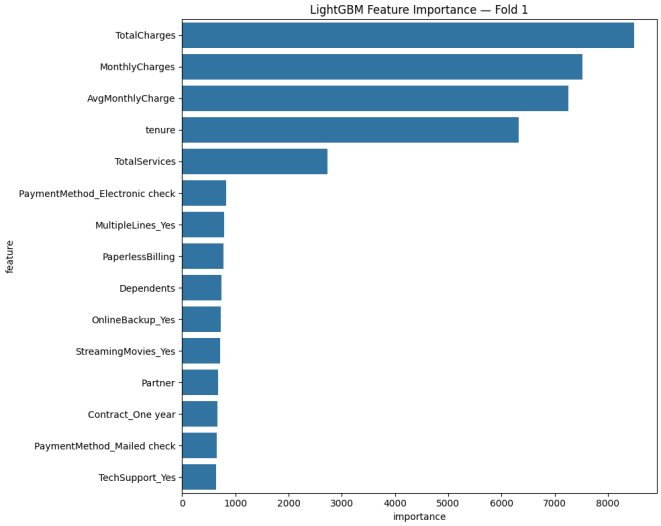


Fig. 7: LightGBM gain-based feature importance for fold 1. The four strongest signals are the numeric charge/tenure attributes and the derived `AvgMonthlyCharge`, which dominate the long tail of one-hot indicators.

CodeQL static analysis is enabled through GitHub's *default setup* (Settings → Code security → Code scanning) rather than via a workflow file; the results surface alongside Gitleaks alerts in the repository's Security tab. The combination of CodeQL, `pip-audit` (dependency vulnerabilities) and Gitleaks (secret hygiene) provides three independent channels of supply-chain assurance.

IX. Lessons Learnt

The following observations crystallised during construction of the pipeline; they are reported here because they were the design-time decisions that proved most consequential.

a) 1. Hand-mirrored preprocessing is the dominant train/serve risk: The serving layer re-implements the training-time feature engineering rather than serialising a scikit-learn `Pipeline` object. This keeps the deployed image small and avoids version-pinning the entire sklearn tree, but it makes the equality of the two implementations a manual invariant. The mitigations adopted—persisting the feature list inside the registered model, validating the API with the same payloads the training code emits, and unit-testing the preprocessor—are defensible but not sufficient. A safer long-term design is to serialise a single sklearn or `FunctionTransformer` pipeline inside the `pyfunc` wrap-

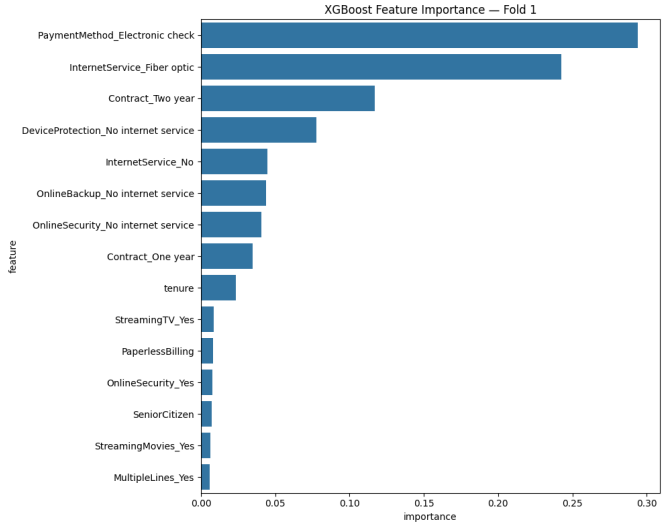


Fig. 8: XGBoost gain-based feature importance for fold 1. The ranking corroborates the LightGBM picture: the same four features dominate but with a different magnitude profile, illustrating why averaging the two learners is preferable to using either one alone.

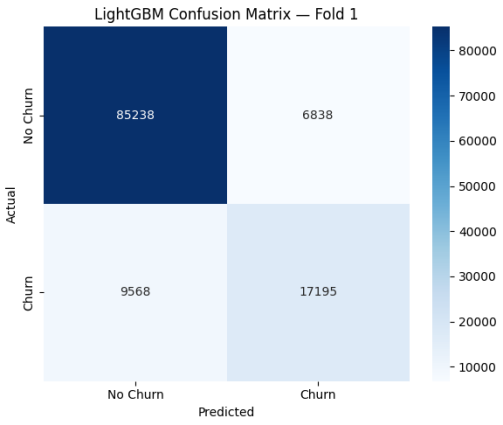
per.

b) 2. Wrapping all fold models in a single registry entity is worth the memory cost: Promoting ten fold models behind a single `@champion` alias makes deployment atomic and rolls forward and back as one unit. The cost is that the deployed container holds ten boosters in memory; for this scale (small, tabular) the trade-off is favourable, but it would not extrapolate to deep-learning ensembles without a more sophisticated serving topology.

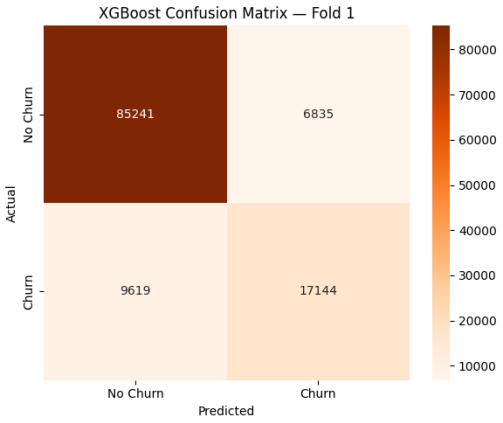
c) 3. The 50 % coverage floor reflects a deliberate scope choice, not laziness: Coverage measurement intentionally excludes `train.py` because training is exercised end-to-end by the `train.yml` workflow rather than by pytest. Lowering the floor to 50 % prevents the gate from rewarding shallow tests written purely to pad the percentage and keeps the rule honest.

d) 4. The service-name versus `localhost` boundary is the most common onboarding stumble: Two URL spaces—intra-network DNS (`mlflow:5001`) and host ports (`localhost:5001`)—coexist throughout the codebase. Any new contributor will encounter this distinction at least once, and it is the source of nearly every "works on my machine" incident on the project. The repository's `CLAUDE.md` now documents the distinction explicitly.

e) 5. A self-hosted Evidently + Postgres + Grafana monitor is viable at this scale: Production teams often default to a managed observability product, but the requirements of an MLOps reference implementation—auditability, offline operation, full control over the schema—are better served by the lightweight self-hosted alternative used here. The eight-panel dashboard, the synthetic drift



(a) LightGBM, fold 1.



(b) XGBoost, fold 1.

Fig. 9: Per-fold confusion matrices logged to MLflow. Classes are imbalanced (roughly 1:3) so the operating threshold is tuned downstream from these matrices.

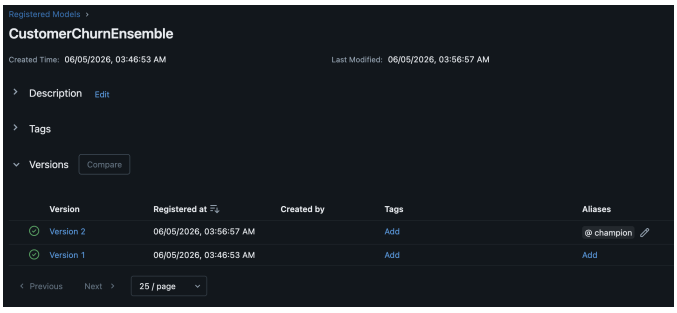


Fig. 10: MLflow registry view. Capture from http://localhost:5001 once a training run has been promoted.

injector and the Postgres schema together account for less than 1 000 lines of code.

f) 6. SLSA v1 attestations are cheap once the build is in Actions: Adding SBOM generation and SLSA provenance to the container build was a one-line addition once the workflow was already publishing to GHCR. The marginal cost is small and the resulting supply-chain story is much

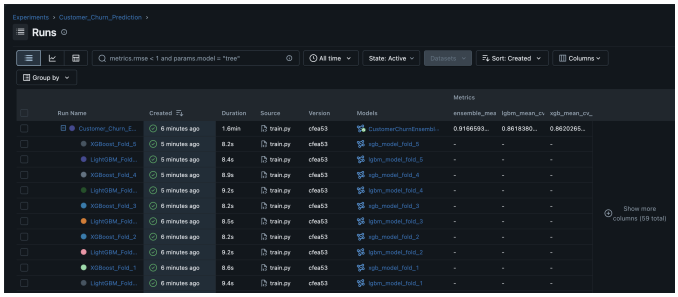


Fig. 11: MLflow run detail. Parent run with nested child runs and per-fold metrics; see Phases 2–4 above.

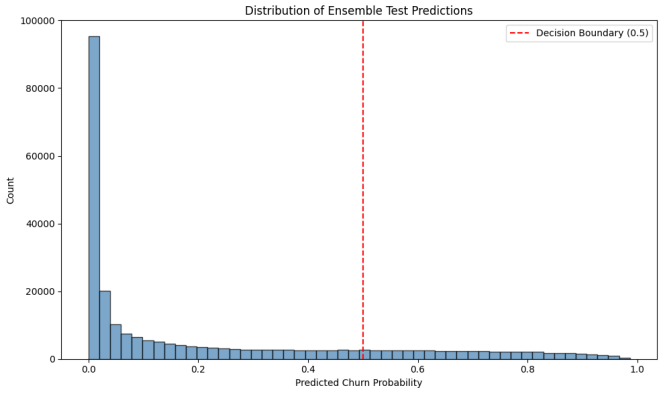


Fig. 12: Distribution of ensemble churn probabilities on the held-out test set. The red dashed line marks the default 0.5 decision boundary.

easier to defend than "trust the image".

X. Future Works

Five directions naturally extend this work:

1) **Workflow orchestration with Prefect.** The presence of an empty `project/prefect-data/` directory in the repository suggests this was already scoped; replacing the ad-hoc batch loop in the monitor with Prefect flows would give first-class scheduling, retries and observability.
2) **Automated champion–challenger gate.** The `@champion` alias is presently moved manually; a gate inside `train.py` that compares the new candidate to the incumbent on a frozen validation slice (with a confidence interval) would close the loop and remove a manual step.
3) **Per-customer explainability via SHAP.** Lundberg and Lee [18] give a unified framework for attributing a prediction to its features; adding SHAP values to the `/predict` response would turn the service from a black-box classifier into a decision-support tool for retention agents.
4) **Alert-driven rollback.** A Grafana alert wired into a small operator could move the `@champion` alias back to the previous version when the drift or quality panels cross a threshold; this would make the loop fully self-healing.
5) **Canary serving.** Running two FastAPI replicas behind a weighted reverse proxy would allow new champion candidates to be A/B-tested in production traffic before fully promoting, eliminating the residual risk of a successful registry promotion that nevertheless degrades live performance.

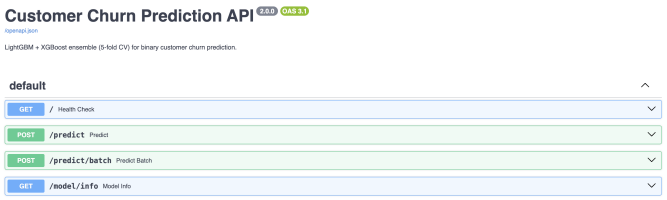


Fig. 13: Auto-generated Swagger UI for the prediction service at http://localhost:8000/docs.

XI. Conclusion

This report has described a complete, reproducible MLOps pipeline for binary customer-churn prediction. The contribution is not algorithmic but architectural: the demonstration that a small set of mature open-source tools—LightGBM and XGBoost for modelling, Optuna for optimisation, MLflow for tracking and registry, FastAPI for serving, Evidently+Postgres+Grafana for monitoring, Docker Compose for orchestration and GitHub Actions for CI/CD—can be composed into a working system that closes the loop from data through training, deployment and observation, with continuous-integration discipline applied end to end. The same architecture would scale, with minor substitutions (e.g., a managed feature store, a Kubernetes deployment, a streaming ingestion path), to a much larger production setting; the patterns—a single-aliased registry entity, train/serve preprocessing parity, replay-based monitoring against a fixed reference, attested container builds—transfer directly.

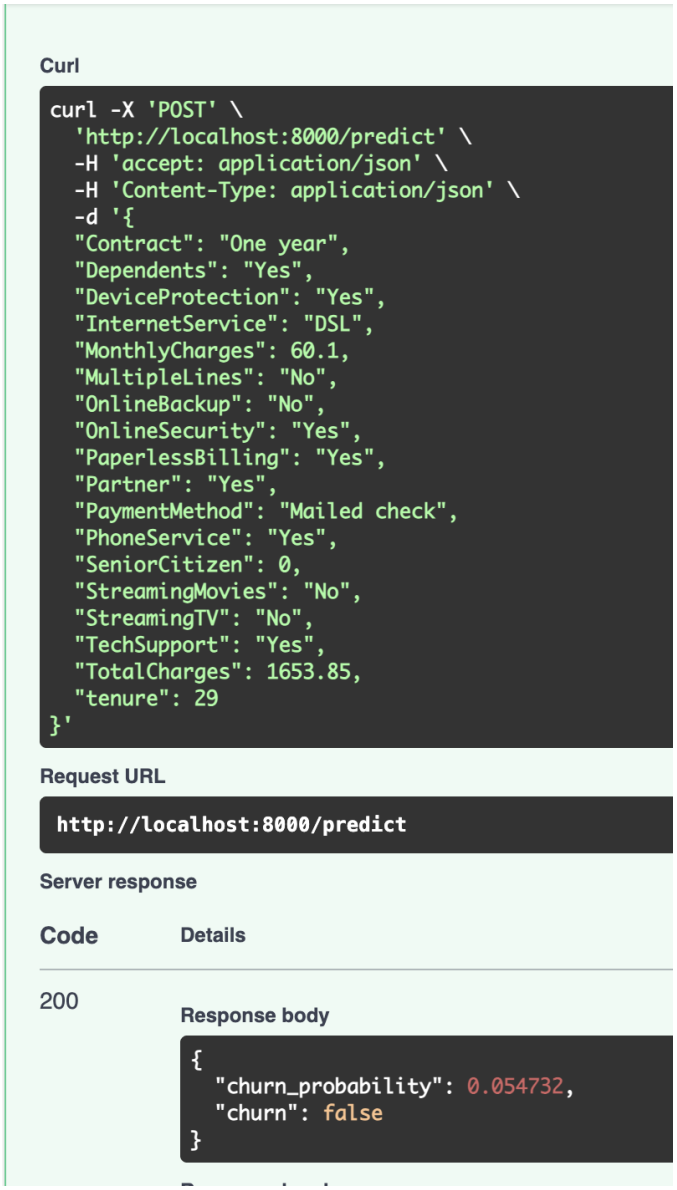


Fig. 14: Example single-customer prediction round-trip via the Swagger “Try it out” panel.

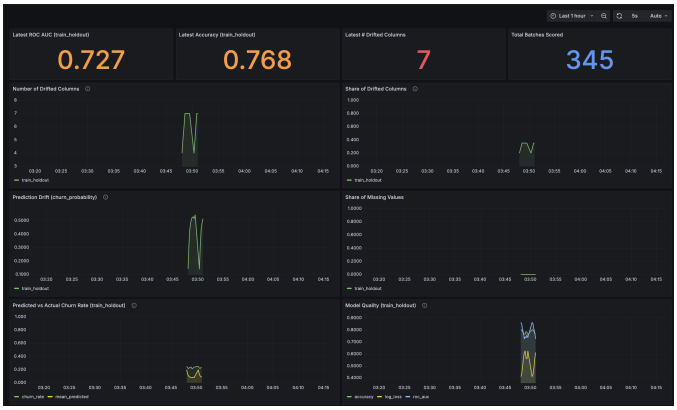


Fig. 15: Grafana monitoring dashboard. Anonymous viewer access is enabled in the provisioning so reviewers can open http://localhost:3000 without authentication.

Fig. 16: Effect of the `DRIFT_MODE=gradual` synthetic drift injection on the dashboard’s drift panels.

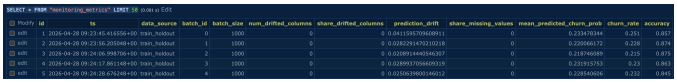


Fig. 17: Adminer view of the metrics table that backs the dashboard. Useful for verifying that the monitor is actually writing rows when the dashboard appears empty.

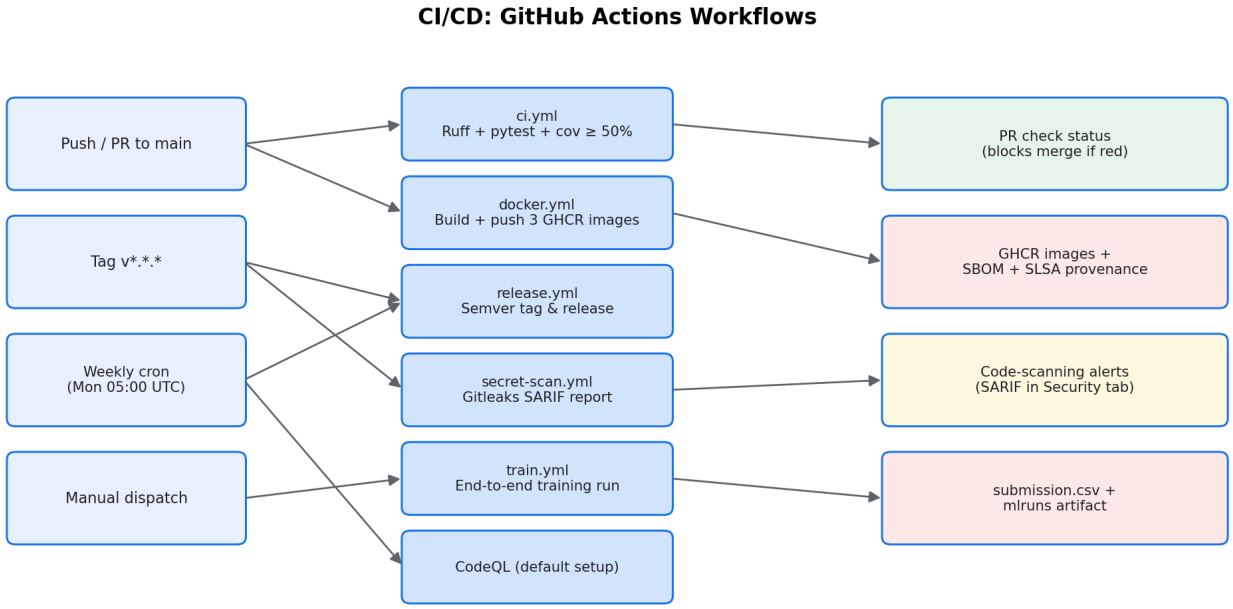


Fig. 18: GitHub Actions topology. Triggers on the left, workflows in the middle, observable outputs on the right.

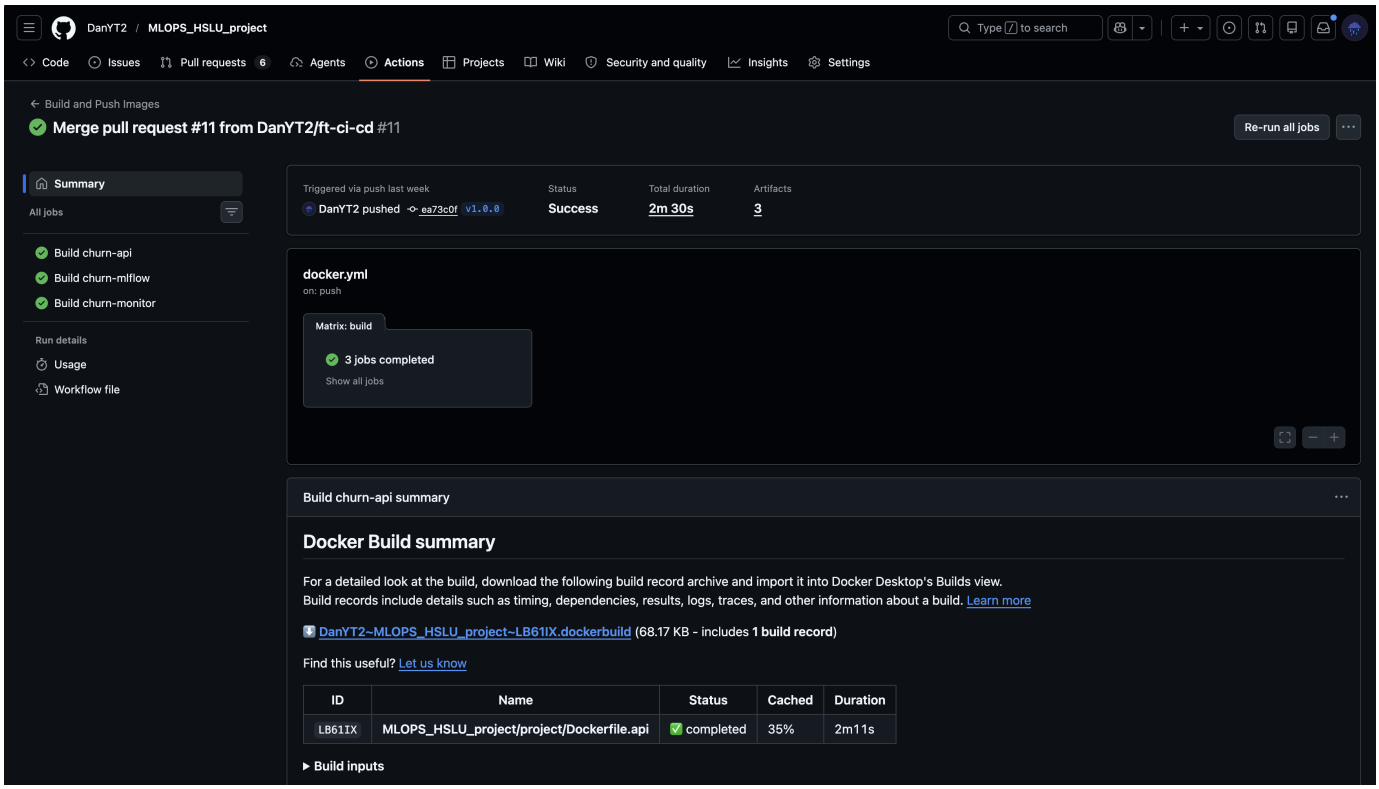


Fig. 19: GitHub Actions run summary for a representative PR.

References

[1] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "LightGBM: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.

[2] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*. ACM, 2016, pp. 785–794.

[3] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A next-generation hyperparameter optimization framework," in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, 2019, pp. 2623–2631.

[4] MLflow Project, "MLflow documentation," https://mlflow.org/docs/latest/index.html, 2024, accessed 2026-06-02.

[5] S. Ramírez, "FastAPI documentation," https://fastapi.tiangolo.com/, 2024, accessed 2026-06-02.

[6] Docker, Inc., "Docker Compose documentation," https://docs.docker.com/compose/, 2024, accessed 2026-06-02.

[7] Evidently AI, "Evidently AI documentation," https://docs.evidentlyai.com/, 2024, accessed 2026-06-02.

[8] GitHub, "GitHub Actions documentation," https://docs.github.com/en/actions, 2024, accessed 2026-06-02.

[9] Open Source Security Foundation, "SLSA v1.0 provenance specification," https://slsa.dev/spec/v1.0/, 2023, accessed 2026-06-02.

[10] D. Sculley, G. Holt, D. Golovin, E. Davydov, T. Phillips, D. Ebner, V. Chaudhary, M. Young, J.-F. Crespo, and D. Dennison, "Hidden technical debt in machine learning systems," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 28, 2015.

[11] A. Paleyes, R.-G. Urma, and N. D. Lawrence, "Challenges in deploying machine learning: A survey of case studies," *ACM Computing Surveys*, vol. 55, no. 6, pp. 1–29, 2022.

[12] W. Verbeke, K. Dejaeger, D. Martens, J. Hur, and B. Baesens, "New insights into churn prediction in the telecommunication sector: A profit-driven data mining approach," *European Journal of Operational Research*, vol. 218, no. 1, pp. 211–229, 2012.

[13] J. Bergstra, R. Bardenet, Y. Bengio, and B. Kégl, "Algorithms for hyper-parameter optimization," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 24, 2011.

[14] E. Breck, S. Cai, E. Nielsen, M. Salib, and D. Sculley, "The ML test score: A rubric for ML production readiness and technical debt reduction," in *IEEE International Conference on Big Data*, 2017, pp. 1123–1132.

[15] N. Polyzotis, S. Roy, S. E. Whang, and M. Zinkevich, "Data lifecycle challenges in production machine learning: A survey," in *SIGMOD Record*, vol. 47, no. 2, 2018, pp. 17–28.

[16] J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, and A. Bouchachia, "A survey on concept drift adaptation," *ACM Computing Surveys*, vol. 46, no. 4, pp. 1–37, 2014.

[17] Grafana Labs, "Grafana documentation," https://grafana.com/docs/grafana/latest/, 2024, accessed 2026-06-02.

[18] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.